A Distributed Multi-key Generation Protocol with a New Complaint Management Strategy

Rym Kalai[1(✉)], Wafa Neji[2], and Narjes Ben Rajeb[3]

[1] LIPSIC Laboratory, Faculty of Sciences of Tunis, University of Tunis El Manar, Tunis, Tunisia
rym.kalai@gmail.com

[2] LIPSIC Laboratory, The General Directorate of Technological Studies, Higher Institute of Technological Studies of Beja, Beja, Tunisia

[3] LIPSIC Laboratory, National Institute of Applied Sciences and Technology, University of Carthage, Tunis, Tunisia

Abstract. Privacy protection is a main goal in the majority of the Blockchain studies. However some dishonest users may abuse from the benefits of this property and the fact of not being identified to do illegal crimes. That is why several researches focus on implementing identity tracing to avoid the flaws related to privacy protection in Blockchain applications.

In this paper, we propose a Distributed Multi-Key Generation (DMKG) protocol without private channels built on the DMKG protocol of the Blockchain Traceable Scheme with Oversight Function (BTSOF) presented in [8].

Our protocol introduces a new strategy to manage complaints between participants that avoids them to publicly reveal the values of their shares of secrets. This new management of complaints and the use of public channels allow a precise identification of malicious participants. We prove that our solution satisfies the security requirements of the Verifiable Multi-Secret Sharing (VMSS) schemes and DMKG protocols.

Keywords: Distributed Multi-Key Generation (DMKG) · Complaints' management strategy · Multi-Secret Sharing (MSS)

1 Introduction

In 1979, Shamir [17] and Blakley [2] introduced the concept of Secret Sharing (SS). It is an important research content of cryptography especially to solve the problem of secret decentralization and to protect secret information from being lost, destructed, inaccessible or falling into the hands of unauthorized third parties [16]. A trusted party called dealer splits the secret into different pieces called shares and distributes them to other parties. Moreover (t, n) threshold cryptosystems allow the dealer to share a secret information between a set of

M. Papadaki et al. (Eds.): EMCIS 2022, LNBIP 464, pp. 150–164, 2023.
https://doi.org/10.1007/978-3-031-30694-5_12

n participants such that only a subgroup of no less than t members among the n participants ($2 \leq t < n$) can cooperate together to recover the secret [20]. The shares of any smaller subset are unable to reveal any information about the secret. The security depends on this unique trusted party known as the dealer who is the only participant that holds the secret key and distributes secret shares to participants. This is the weak point of the system aiming to protect the secret information and guarantee that no party owns it [1]. Distributed Key Generation (DKG) protocols offer the solution. In 1991, Pedersen [14] presented the first DKG protocol with no trusted party. It involves many participants to cooperate together to generate and share the secret key. All honest participants have a valid share of a unique secret key. There is n secret sharing processes: Each participant plays the role of a dealer on a single sharing process. Yet, DKG protocols permit to share only one secret among the participants. Multi-Secret Sharing (MSS) scheme was the key to solve this problem. In a MSS scheme, the dealer distributes several secrets among the participants at the same time. In addition, in (t, n, l) threshold multi-secret sharing schemes, at least t or more participants can easily pool their shares and reconstruct l secrets at the same time [19]. Once again, it ends up with a single party holding all the secrets. Thus, this is the point of failure of the whole scheme. To overcome this problem, the idea is to do as in the DKG protocols except that the participants handle t several secrets at the same time. It is the equivalent of having n parallel executions of the MSS scheme. This is called Distributed Multi-Key Generation (DMKG) protocol introduced by Tianjun Ma *et al.* in [8] in 2020. However, not all participants are honest. Some may cheat and distribute invalid shares. The protocol must detect the cheating either by using an expensive public verification which affects the scalability of the DMKG protocol, or by using a complaint management strategy that allows participants to complain if they receive invalid shares [3,6,12,14]. The issue is that this strategy forces participants to reveal their shares in order to prove their honesty. In addition, a malicious participant may cheat and complain about receiving an invalid share to eliminate an honest participant. The challenge is to detect dishonest participants and to prove their honesty without revealing their shares [11,15].

In this contribution, we present our DMKG protocol which relies on the *Disputation* strategy proposed in [11] to manage complaints between participants. This strategy avoids participants to reveal their shares and disqualifies the malicious ones. However, it is limited to a unique secret sharing. We extend it in our DMKG protocol to suit the multi secret sharing context.

Note that we use only public channels and that all the shares of secrets are encrypted. The construction of our DMKG protocol is built on the DMKG protocol presented in [8]. The multi-secret sharing scheme is based on the verifiable Pedersen's secret sharing scheme [13] and leverage the Franklin and Yung multi-secret sharing scheme [5].

This paper is organized as follows: We start by presenting the related work of our research namely the multi-secret sharing scheme, the most important DMKG protocols and the Blockchain Traceable Scheme with Oversight Function

(BTSOF) defined in [8]. Then, we detail our new DMKG protocol with a new complaint management strategy based on BTSOF's DMKG protocol. Next, we prove the security of our protocol. Finally, we present a short comparison between our protocol and BTSOF's DMKG protocol.

2 State of the Art

The term Distributed Multi-Key Generation (DMKG) protocol was introduced by Tianjun Ma *et al.* in [8] in 2020 to describe a distributed and parallel executions of a MSS scheme. As mentioned in the previous section, the DMKG protocol is a main part of distributed encryption system. It solves the problem of the single trusted entity in MSS schemes. Each participant will play the role of the dealer. It involves the participants to cooperate together and share multiple secrets to generate a private and public keys. At the end, only an authorized subset of them can reconstruct the private key. The DMKG protocol proposed in [8] was used to design a Blockchain Traceable Scheme with Oversight Function (BTSOF). In 2021, Tianjun Ma *et al.* [10] improved the scheme and proposed a traceable scheme that is suitable for consortium Blockchain with leverages of the proposed DMKG protocol. On the other hand, to guarantee uniform distribution of the generated keys, the DMKG proposed by Tianjun Ma *et al.* in [8] is built on the Gennaro's DKG protocol [6]. This last, has proved that the presence of malicious participants disrupts the distribution of the generated keys and that all protocols based on Pedersen's DKG do not assure a uniform distribution. The proposed protocol uses private channels between participants and requires extra communications to deal with the complaints between participants. This increases the complexity of the protocol and makes it difficult to put into practice. Also, the private channels make it hard to determine the malicious participants.

In this paper, we propose a new DMKG protocol with a (t, n) threshold multi secret sharing (TMSS) scheme based on the Franklin and Yung MSS scheme with no private channels. The MSS scheme is simultaneous: all the secrets are reconstructed at the same time in only one stage [7]. Moreover, it is a verifiable scheme based on Pedersen's Verifiable Secret Sharing (VSS). In addition, our DMKG protocol offers a new complaints management strategy that avoids participants' shares revealing.

3 DMKG Protocol of BTSOF

This section presents the DMKG protocol of Blockchain Traceable Scheme with Oversight Function (BTSOF) introduced in [8].

3.1 Notation

Let p and q denote two large prime numbers, such that $q|(p-1)$. $\mathbb{Z}_p$ denotes a group of order p and $\mathbb{Z}_q$ denotes a group of order q. By default the exponential operation performs modulo p operation. For example, g^x denotes $g^x mod p$, where $g \in \mathbb{Z}_p$ and $x \in \mathbb{Z}_q$. (pk_{tra}, sk_{tra}) denotes the traceable public-private key pair.

3.2 Context

Some dishonest users can commit crimes by taking advantage of the fact of not being identified. Several researches focus on implementing identity tracing to avoid the flaws related to privacy protection in Blockchain applications. In 2020, Tianjun Ma *et al.* presented a Blockchain traceable scheme denoted SkyEye [9]. It allows the regulator to trace the users' identities of the Blockchain data. However, there are no restrictions and oversight measures. The regulator can arbitrarily trace the data. In [8], Tianjun Ma *et al.* proposed an improvement of SkyEye in BTSOF. The regulator must obtain the consent of the committee to enable tracing. He can trace one data, multiple data or data in multiple period. In this paper, we focus only on tracing all data of the T period denoted $data^T$. The committee has n participants $P_1, ..., P_n$. The traceable public-private key pair (pk_{tra}^T, sk_{tra}^T) is generated by the committee in a T period. The private key sk_{tra}^T is sent by the committee to the regulator. The tracing process is as follows: Let u denotes a user, for each user's data noted $data_u$ in $data^T$, the regulator gets the chameleon hash public key set PK_C by decrypting each ciphertext of chameleon hash public key in $data_u$ using the private key sk_{tra}^T. He can obtain the users' true identity set ID corresponding to $data_u$ by searching the database according to PK_C. The traceable public-private key pair is generated by the committee through the DMKG protocol. The protocol is based on the DKG protocol proposed by Gennaro *et al.* [6]. It uses a non-interactive VMSS scheme for the Cramer-Shoup public key encryption scheme [4]. The VMSS is formed from the combination of the Franklin and Yung MSS scheme and Pedersen's VSS.

3.3 Communication and Adversary Model

They assume that there is a set of n probabilistic polynomial time participants $P_1, P_2, \ldots, P_n$ in a fully synchronous network. All participants have a common broadcast channel, and there are private channels between participants.

The adversary A is static. At the beginning of the protocol, he chooses the corrupted participants. From the n participants, he can corrupt at most $(t-1)$, such that $(t-1) < \frac{n}{2}$.

3.4 The DMKG Protocol

The traceable public-private key pair is generated by the committee through the DMKG protocol. In this paper, we focus specially on the generation of the traceable private key. We simplify the notations and note sk the traceable private key and pk the traceable public key. The protocol consists on n parallel executions of the VMSS scheme defined in [8]. The generation of sk is done in three phases as follows:

Initialization Phase. A trusted authority has chosen $g_1, h_1, g_2, h_2 \in \mathbb{Z}_p$, where $h_1 = {g_1}^{\gamma_1}$ and $h_2 = {g_2}^{\gamma_2}$ for $\gamma_1, \gamma_2 \in \mathbb{Z}_q$. The protocol aims to generate the private key $sk = (x_1, x_2, y_1, y_2, z)$.

Distribution and Verification Phase. For $i = 1, ..., n$, the committee member P_i is the dealer and performs the following steps:

Step 2.a P_i chooses at random $x_{1i}, x_{2i}, y_{1i}, y_{2i}, z_i \in \mathbb{Z}_q$ and $\beta_{0i}, \beta_{1i}, \beta_{2i}, \beta_{3i}, \beta_{4i} \in \mathbb{Z}_q$.

Step 2.b P_i broadcasts $E_{1i} = {g_1}^{x_{1i}}.{h_1}^{\beta_{1i}}$, $E_{2i} = {g_1}^{y_{1i}}.{h_1}^{\beta_{2i}}$, $E_{3i} = {g_2}^{x_{2i}}.{h_2}^{\beta_{3i}}$, $E_{4i} = {g_2}^{y_{2i}}.{h_2}^{\beta_{4i}}$, $E_{5i} = {g_1}^{z_i}$

Step 2.c P_i randomly chooses 6 polynomials $f(x), f'(x), g(x), g'(x), h(x)$ et $h'(x)$ of degree t such that $f_i(x) = a_{i0} + a_{i1}x + ... + a_{it}x^t$, $f'_i(x) = b_{i0} + b_{i1}x + ... + b_{it}x^t$, $g_i(x) = a'_{i0} + a'_{i1}x + ... + a'_{it}x^t$, $g'_i(x) = b'_{i0} + b'_{i1}x + ... + b'_{it}x^t$, $h_i(x) = a''_{i0} + a''_{i1}x + ... + a''_{it}x^t$, $h'_i(x) = b''_{i0} + b''_{i1}x + ... + b''_{it}x^t$ where $f_i(-1) = x_{1i}$, $f_i(-2) = y_{1i}$, $f'_i(-1) = \beta_{1i}$, $f'_i(-2) = \beta_{2i}$, $g_i(-1) = x_{2i}$, $g_i(-2) = y_{2i}$, $g'_i(-1) = \beta_{3i}$, $g'_i(-2) = \beta_{4i}$, $h_i(0) = z_i = a''_{i0}$, and $h'_i(0) = \beta_{0i} = b''_{i0}$. The distribution process of z_i uses the Pedersen-VSS scheme [13].

Step 2.d P_i broadcasts : for $k = 0, ..., t$ $CM_{ik} = {g_1}^{a_{ik}}.{h_1}^{b_{ik}}.{g_2}^{a'_{ik}}.{h_2}^{b'_{ik}}$ and $cm_{ik} = {g_1}^{a''_{ik}}.{h_1}^{b''_{ik}}$ where $cm_{i0} = {g_1}^{a''_{i0}}.{h_1}^{b''_{i0}} = {g_1}^{z_i}.{h_1}^{\beta_{0i}}$.

Step 2.e For each $i = 1, ..., n$, for $\tau = 1, 2$, for each $j = 1, ..., n$ P_j checks:

$$E_{\tau i}.E_{(\tau+2,i)} \stackrel{?}{=} \prod_{k=0}^{t} (CM_{ik}^{(-\tau)^k}) \tag{1}$$

If it fails for an index i, then P_i is a dishonest participant and is disqualified, so $i \notin Q_{temp}$ where Q_{temp} denotes the temporarily set of qualified participants.

Step 2.f For $i = 1, ..., n$, P_i computes the shares $sf_{ij} = f_i(j)$, $sf'_{ij} = f'_i(j)$, $sg_{ij} = g_i(j)$, $sg'_{ij} = g'_i(j)$, $sh_{ij} = h_i(j)$ and $sh'_{ij} = h'_i(j)$. Then, P_i sends secretly the set of shares $(sf_{ij}, sf'_{ij}, sg_{ij}, sg'_{ij}, sh_{ij}, sh'_{ij})$ to each participant P_j.

Step 2.g Each participant P_j checks for each $i \in Q_{tem}$:

$$\begin{cases} ({g_1}^{sf_{ij}}.{h_1}^{sf'_{ij}}.{g_2}^{sg_{ij}}.{h_2}^{sg'_{ij}}) \stackrel{?}{=} \prod_{k=0}^{t} (CM_{ik})^{j^k} \\ {g_1}^{sh_{ij}}.{h_1}^{sh'_{ij}} \stackrel{?}{=} \prod_{k=0}^{t} (cm_{ik})^{j^k} \end{cases} \tag{2}$$

If it fails, then the participant P_j complaints against P_i. Otherwise P_j accepts the set of shares.

Step 2.h If the participant P_i, for $i \in Q_{tem}$, received a complaint from P_j then P_i reveals and broadcasts the set of shares that satisfy Eq. 2.

Step 2.i A participant P_i, $(i = 1, ..., n)$, is marked as disqualified if the number of complaints against P_i is superior than $(t - 1)$ in the *Step 2.g* or if the values broadcasted by P_i in *Step 2.h* do not satisfy Eq. 2.

Step 2.j Each participant P_i, $i \in Q_{tem}$, constructs a set of qualified participants Q_{final}. It contains the participants that are not disqualified in the *Step 2.e* and *Step 2.i*.

Recovery Phase. Each participant P_i, $(i = 1, ..., n)$ computes the shares : $sf_i = \sum_{j \in Q_{final}} sf_{ji} modq$, $sf'_i = \sum_{j \in Q_{final}} sf'_{ji} modq$, $sg_i = \sum_{j \in Q_{final}} sg_{ji} modq$, $sg'_i = \sum_{j \in Q_{final}} sg'_{ji} modq$, $sh_i = \sum_{j \in Q_{final}} sh_{ji} modq$, $sh'_i = \sum_{j \in Q_{final}} sh'_{ji} modq$. The private key sk is not computed. Each committee member P_i with $i = 1...n$, has the traceable private key component $(x_{1i}, x_{2i}, y_{1i}, y_{2i}, z_i)$ so the private key $sk = (x_1 = \sum_{i \in Q_{final}} x_{1i}\ modq$, $x_2 = \sum_{i \in Q_{final}} x_{2i}\ modq$, $y_1 = \sum_{i \in Q_{final}} y_{1i}\ modq$, $y_2 = \sum_{i \in Q_{final}} y_{2i}\ modq$, $z = \sum_{i \in Q_{final}} z_i\ modq)$.

4 Proposed DMKG Protocol

After presenting the BTSOF's DMKG protocol, an attention should be drawn to security flaws encountered. First, the set of shares is publicly revealed when there is a complaint about a participant: when the participant P_j finds that the set of shares sent by P_i did not verify Eq. 1, then he complaints about P_i. This forces P_i to reveal publically the set of shares that satisfy the equation. In addition, the use of private point-to-point channels for secretly distributing the shares, hides dishonest participants: is it the dealer who distributed an invalid share or the participant who lies and claims to have received an invalid share? The protocol fails to identify who.

In this section, we present a DMKG protocol with only public channels. There is no private communication between participants. All the shares are encrypted. For the complaint handling between the participants, our protocol is characterized by a phase named *Complaint management* like [11] protocol. It solves participants' complaints and correctly identifies malicious participants without revealing any information on honest participants' shares. Our protocol suits the MSS context unlike [11].

4.1 Communication and Adversary Model

We consider the same adversary and communication model as [8]. However, in our contribution, there are no private channels between participants. All information including the set of shares are distributed through the public channel.

4.2 The Protocol

In our DMKG protocol, we separate the verification phase from the distribution phase. The generation of the private key $sk = (x_1, x_2, y_1, y_2, z)$ is done in five phases (Initialization, Distribution, Verification, Complaint management, Recovery).

Initialization Phase. During the initialization phase, we keep the same assumptions as the DMKG protocol in [8]. In addition, each participant P_i selects a private key sk_{P_i} and publishes a public key $pk_{P_i} = g^{sk_{P_i}}$ where $sk_{P_i} \in \mathbb{Z}_q$ and g is a generator of prime order group. The public key pk_{P_i} will be used to encrypt and send the shares.

Distribution Phase. We keep the same first steps (from *Step 2.a* to *Step 2.d*) in BTSOF' DMKG protocol except for the *Step 2.b*. In our protocol, all verifications are done after the end of the distribution phase.

Step 2.e Like BTSOF, for $j = 1, ..., n$, P_i computes the shares $sf_{ij} = f_i(j)$, $sf'_{ij} = f'_i(j)$, $sg_{ij} = g_i(j)$, $sg'_{ij} = g'_i(j)$, $sh_{ij} = h_i(j)$, $sh'_{ij} = h'_i(j)$. For $j = 1, ..., n$, P_i actually has the set of shares $(sf_{ij}, sf'_{ij}, sg_{ij}, sg'_{ij}, sh_{ij}, sh'_{ij})$ to be sent to P_j.

Step 2.f Unlike [8], to avoid using private channels between participants, our proposed idea is to encrypt the shares with a simple encryption function and then broadcast them instead of securely sending them to the recipient. In [11], Neji *et al.* encrypt the shares of a secret as follow: $E_{ij} = s_{ij}(pk_{P_j})^{s_{ij}} = s_{ij}(g^{sk_{P_j}})^{s_{ij}}$, where s_{ij} is the share sent from participant P_i to participant P_j and g is a generator of prime order group. As a reminder, each committee member P_i has a public-private key pair (pk_{P_i}, sk_{P_i}). These keys will be used to encrypt the set of shares computed by P_i following the encryption function in [11]. For $j = 1, ..., n$, P_i computes: $Enc_{sf_{ij}} = sf_{ij}(pk_{P_j})^{sf_{ij}} = sf_{ij}(g^{sk_{P_j}})^{sf_{ij}}$, $Enc_{sf'_{ij}} = sf'_{ij}(pk_{P_j})^{sf'_{ij}} = sf'_{ij}(g^{sk_{P_j}})^{sf'_{ij}}$, $Enc_{sg_{ij}} = sg_{ij}(pk_{P_j})^{sg_{ij}} = sg_{ij}(g^{sk_{P_j}})^{sg_{ij}}$, $Enc_{sg'_{ij}} = sg'_{ij}(pk_{P_j})^{sg'_{ij}} = sg'_{ij}(g^{sk_{P_j}})^{sg'_{ij}}$, $Enc_{sh_{ij}} = sh_{ij}(pk_{P_j})^{sh_{ij}} = sh_{ij}(g^{sk_{P_j}})^{sh_{ij}}$, $Enc_{sh'_{ij}} = sh'_{ij}(pk_{P_j})^{sh'_{ij}} = sh'_{ij}(g^{sk_{P_j}})^{sh'_{ij}}$. The participant P_i sends the set of encrypted shares $(Enc_{sf_{ij}}, Enc_{sf'_{ij}}, Enc_{sg_{ij}}, Enc_{sg'_{ij}}, Enc_{sh_{ij}}, Enc_{sh'_{ij}})$ ***publicly*** to P_j for $j = 1, ..., n$.

Verification Phase

Step 3.a Each participant P_j decrypts the set of the shares received from the participant P_i by using the private key sk_{P_j}. $sf_{ij} = Enc_{sf_{ij}}[(g^{sf_{ij}})^{sk_{P_j}}]^{-1}$, $sf'_{ij} = Enc_{sf'_{ij}}[(g^{sf'_{ij}})^{sk_{P_j}}]^{-1}$, $sg_{ij} = Enc_{sg_{ij}}[(g^{sg_{ij}})^{sk_{P_j}}]^{-1}$, $sg'_{ij} = Enc_{sg'_{ij}}[(g^{sg'_{ij}})^{sk_{P_j}}]^{-1}$, $sh_{ij} = Enc_{sh_{ij}}[(g^{sh_{ij}})^{sk_{P_j}}]^{-1}$, $sh'_{ij} = Enc_{sh'_{ij}}[(g^{sh'_{ij}})^{sk_{P_j}}]^{-1}$. At the end of this step, for each $i \in Q_{tem}$, each P_j has a set of shares $(sf_{ij}, sf'_{ij}, sg_{ij}, sg'_{ij}, sh_{ij}, sh'_{ij})$ that needs to be verified.

Step 3.b From the moment that the participant P_j has the set of decrypted shares, we could now verify Eq. 2 as in the *Step 2.g* of BTSOF. If it fails, then the participant P_j complaints against P_i. This complaint will be examined in the *Complaint management* phase. Otherwise, P_j accepts the set of shares.

Complaint Management Phase. This phase is executed whenever there is a complaint from a participant P_j against a participant P_i. The complaint management strategy is inspired by the *Disputation* idea presented in [11,18] and adapted to MSS context. First of all, a participant P_i is marked as disqualified $(i = 1, ..., n)$ if the number of complaints against P_i is superior than $(t-1)$ in the *Step 3.b*. Second, all participants that are not disqualified in the *Step 3.b* will be considered as temporarily qualified and $\in Q_{temp}$. Finally, if a participant P_i is already disqualified, then all the complaints against P_i are accepted. Else, both participants P_i and P_j have to prove their honesty to all the other participants. Let's note Q the set of participants where $Q = \{Q_{temp} \setminus \{P_i, P_j\}\}$.

Step 4.a P_j chooses at random $S'_j = \{a_j, b_j, a'_j, b'_j, a''_j, b''_j\}$ and publishes $S''_j = \{{g_1}^{a_j}, {g_2}^{a'_j}, {g_1}^{a''_j}, {h_1}^{b_j}, {h_2}^{b'_j}, {h_1}^{b''_j}\}$

Step 4.b Each P_k with $k \in Q$ chooses at random $S'_k = \{a_k, b_k, a'_k, b'_k, a''_k, b''_k\}$ and publishes $S''_k = \{{g_1}^{a_k}, {g_2}^{a'_k}, {g_1}^{a''_k}, {h_1}^{b_k}, {h_2}^{b'_k}, {h_1}^{b''_k}\}$

Step 4.c P_i computes and publishes: $\lambda_1 = sf_{ij}[{g_1}^{a_j}.\prod_{k=1;P_k\in Q}^{n} {g_1}^{a_k}]^{sf_{ij}}$, $\lambda'_1 = sg_{ij}[{g_2}^{a'_j}.\prod_{k=1;P_k\in Q}^{n} {g_2}^{a'_k}]^{sg_{ij}}$, $\lambda''_1 = sh_{ij}[{g_1}^{a''_j}.\prod_{k=1;P_k\in Q}^{n} {g_1}^{a''_k}]^{sh_{ij}}$, $\lambda_2 = sf_{ij}({g_1}^{a_j})^{sf_{ij}}$, $\lambda'_2 = sg_{ij}({g_2}^{a'_j})^{sg_{ij}}$, $\lambda''_2 = sh_{ij}({g_1}^{a''_j})^{sh_{ij}}$, $\gamma 1 = sf'_{ij}[{h_1}^{b_j}.\prod_{k=1;P_k\in Q}^{n} {h_1}^{b_k}]^{sf'_{ij}}$, $\gamma' 1 = sg'_{ij}[{h_2}^{b'_j}.\prod_{k=1;P_k\in Q}^{n} {h_2}^{b'_k}]^{sg'_{ij}}$, $\gamma'' 1 = sh'_{ij}[{h_1}^{b''_j}.\prod_{k=1;P_k\in Q}^{n} {h_1}^{b''_k}]^{sh'_{ij}}$, $\gamma 2 = sf'_{ij}({h_1}^{b_j})^{sf'_{ij}}$, $\gamma' 2 = sg'_{ij}({h_2}^{b'_j})^{sg'_{ij}}$, $\gamma'' 2 = sh'_{ij}({h_1}^{b''_j})^{sh'_{ij}}$.

Step 4.d Each participant P_k with $k \in Q$ publishes his set S'_k. If any participant cheats by publishing an invalid set which does not correspond to S''_k, then he will be disqualified and removed from Q, and the process resumes at *Step 4.b*.

Step 4.e Each participant P_k with $k \in Q$ computes : $\alpha = \frac{\lambda_1}{\lambda_2}$, $\alpha' = \frac{\lambda'_1}{\lambda'_2}$, $\alpha'' = \frac{\lambda''_1}{\lambda''_2}$, $\beta = \frac{\gamma_1}{\gamma_2}$, $\beta' = \frac{\gamma'_1}{\gamma'_2}$, $\beta'' = \frac{\gamma''_1}{\gamma''_2}$ and $r = \sum_{k=1}^{n} a_k$, $r' = \sum_{k=1}^{n} a'_k$, $r'' = \sum_{k=1}^{n} a''_k$, $t = \sum_{k=1}^{n} b_k$, $t' = \sum_{k=1}^{n} b'_k$, $t'' = \sum_{k=1}^{n} b''_k$. Then, P_k computes: $\alpha^{\frac{1}{r}} = {g_1}^{sf_{ij}}$, ${\alpha'}^{\frac{1}{r'}} = {g_2}^{sg_{ij}}$, ${\alpha''}^{\frac{1}{r''}} = {g_1}^{sh_{ij}}$ and $\beta^{\frac{1}{t}} = {h_1}^{sf'_{ij}}$, ${\beta'}^{\frac{1}{t'}} = {h_2}^{sg'_{ij}}$, ${\beta''}^{\frac{1}{t''}} = {h_1}^{sh'_{ij}}$. P_k verifies

$$\begin{cases} \alpha^{\frac{1}{r}}.\beta^{\frac{1}{t}}.{\alpha'}^{\frac{1}{r'}}.{\beta'}^{\frac{1}{t'}} \stackrel{?}{=} \prod_{k=0}^{t} (CM_{ik})^{j^k} \\ {\alpha''}^{\frac{1}{r''}}.{\beta''}^{\frac{1}{t''}} \stackrel{?}{=} \prod_{k=0}^{t} (cm_{ik})^{j^k} \end{cases} \tag{3}$$

If it fails, then the secret set $(sf_{ij}, sf'_{ij}, sg_{ij}, sg'_{ij}, sh_{ij}, sh'_{ij})$ is not correct. P_k concludes that P_i is dishonest and the protocols ends. Otherwise the protocol continues.

Step 4.f Each participant $P_j \notin Q$ computes: $sf_{ij} = \frac{\lambda_2}{(g_1{}^{sf_{ij}})^{a_j}}$, $sg_{ij} = \frac{\lambda'_2}{(g_2{}^{sg_{ij}})^{a'_j}}$, $sh_{ij} = \frac{\lambda''_2}{(g_1{}^{sh_{ij}})^{a''_j}}$ and $sf'_{ij} = \frac{\gamma_2}{(h_1{}^{sf'_{ij}})^{b_j}}$, $sg'_{ij} = \frac{\gamma'_2}{(h_2{}^{sg'_{ij}})^{b'_j}}$, $sh'_{ij} = \frac{\gamma''_2}{(h_1{}^{sh'_{ij}})^{b''_j}}$. P_j verifies if these values check Eq. 2. If it checks, the participant P_j has a valid set of shares. In this case, P_j sends a confirmation to all participants in Q and the protocol ends. Else, P_j sends the set S'_j to all participants in Q.

Step 4.g Each participant $P_k \in Q$ checks if the set S'_j sent by P_j is valid. This means that P_k verifies if the values $(g_1{}^{a_j}, g_2{}^{a'_j}, g_1{}^{a''_j}, h_1{}^{b_j}, h_2{}^{b'_j}, h_1{}^{b''_j})$ published by P_j in *step 4.a* correspond to the values in the set S'_j published by P_j in *step 4.f*. If it fails, P_k concludes that P_j lied and that he is dishonest and the protocol ends. Otherwise, the protocol continues.

Step 4.h Each $P_k \in Q$ computes $(g_1{}^{sf_{ij}})^{a_j}$, $(g_2{}^{sg_{ij}})^{a'_j}$, $(g_1{}^{sh_{ij}})^{a''_j}$, $(h_1{}^{sf'_{ij}})^{b_j}$, $(h_2{}^{sg'_{ij}})^{b'_j}$, $(h_1{}^{sh'_{ij}})^{b''_j}$ and $e_1 = \frac{\lambda_2}{(g_1{}^{sf_{ij}})^{a_j}}$, $e'_1 = \frac{\lambda'_2}{(g_2{}^{sg_{ij}})^{a'_j}}$, $e''_1 = \frac{\lambda''_2}{(g_1{}^{sh_{ij}})^{a''_j}}$, $e_2 = \frac{\gamma_2}{(h_1{}^{sf'_{ij}})^{b_j}}$, $e'_2 = \frac{\gamma'_2}{(h_2{}^{sg'_{ij}})^{b'_j}}$, $e''_2 = \frac{\gamma''_2}{(h_1{}^{sh'_{ij}})^{b''_j}}$. P_k verifies if the equations in *Step 4.c* are correct with : $e_1 = sf_{ij}$, $e'_1 = sg_{ij}$, $e''_1 = sh_{ij}$, $e_2 = sf'_{ij}$, $e'_2 = sg'_{ij}$, $e''_2 = sh'_{ij}$. If there is equality, then P_k concludes that P_j lied, else, P_i lied.

Step 4.i Each $P_k \in Q$ broadcasts the result of the previous steps by indicating whether the dishonest participant is P_i or P_j.

Step 4.j The dishonest participant is identified by counting the number of votes for P_i or P_j and selecting the maximum. Thus, thanks to the *Complaint management* phase, dishonest participants are identified and disqualified and we can construct $QUAL$ the set of non disqualified participants.

Recovery Phase. Each participant $P_i \in QUAL$ computes the shares: $sf_i = \sum_{j \in QUAL} sf_{ji} mod q$, $sf'_i = \sum_{j \in QUAL} sf'_{ji} mod q$, $sg_i = \sum_{j \in QUAL} sg_{ji} mod q$, $sg'_i = \sum_{j \in QUAL} sg'_{ji} mod q$, $sh_i = \sum_{j \in QUAL} sh_{ji} mod q$, $sh'_i = \sum_{j \in QUAL} sh'_{ji} mod q$.

As a result of this approach, P_i obtains a set of qualified participants $QUAL$ and holds for $j \in QUAL$ the values $f_j(i)$, $g_j(i)$, and $h_j(i)$.

5 Security

Our DMKG protocol respects the same security requirements in [8] which are secrecy and correctness for distributed multi-key generation in the Cramer-Shoup encryption scheme [4] in the presence of an adversary A that corrupts at most $(t-1)$ participants for any $(t-1) < \frac{n}{2}$.

5.1 Security Requirements

Correctness. The correctness of the protocol includes the following elements:

C1 Any set of $(t+1)$ shares provided by honest participants defines the same unique secret key $sk = (x_1, x_2, y_1, y_2, z)$.

C2 There is an efficient algorithm that has as input n shares submitted by participants and outputs the unique secret key sk even if at most $(t-1)$ invalid shares are submitted by dishonest participants.

C3 All honest participants have the same public key $pk = (c_1, c_2, c_3) = ({g_1}^{x_1}, {g_2}^{x_2}, {g_1}^{y_1}, {g_2}^{y_2}, {g_1}^{z})$, where $sk = (x_1, x_2, y_1, y_2, z)$ is the unique secret key guaranteed by (C1).

C4 The secret key $sk = (x_1, x_2, y_1, y_2, z)$ is uniformly distributed in $\mathbb{Z}_q$.

Secrecy. Since the shared private key sk is a confidential information that should not be found by a party who is unauthorized to know it, the DMKG protocol verifies the secrecy requirement if the adversary gets nothing about sk, or any information on sk except for the pubic key pk. We use the same simulation in [8] to formally express the secrecy of the DMKG and to prove that an adversary A cannot compute the value of a secret key sk even if he can corrupt at most $(t-1)$ participants and can use their secret information. For each probabilistic polynomial-time adversary, a simulator takes as input the public key pk and produces as an output a distribution that is indistinguishable from the adversary's view in the real run of the DMKG protocol that outputs the public key pk. This dishonest participant cannot compute the value of the secret key sk.

5.2 Security Proofs

In this section, we prove that our DMKG protocol satisfies the security requirements presented previously in 5. First of all, we need to present the following lemma about the Pedersen VSS scheme [6,13] and the VMSS scheme in [8].

Lemma 1. *In the presence of an adversary that corrupts at most t participants, any* $(t+1)$ *shares of the honest participants can reconstruct the secret s. If the dealer is honest, then all shares held by honest participants can interpolate to a unique polynomial of degree t.*

Lemma 2. *In the presence of an adversary that corrupts at most* $(t-l+1)$ *participants, any* $(t+1)$ *shares of the honest participants can reconstruct the secret set* $S = s_1, \ldots, s_l$. *If the dealer is honest, then all shares held by honest participants can interpolate to a unique polynomial of degree t.*

Correctness Proof.

Theorem 1. *Our DMKG protocol satisfies the security requirement (C1). All shares submitted by a set of honest participants define the same private key sk.*

Proof. At the end of the *Complaint Management* phase, if the participant P_i is not disqualified, which means that $P_i \in QUAL$ and that as a dealer, he has correctly performed the Pedersen-VSS and VMSS scheme. We rely on Lemma 1 and Lemma 2. Then, all honest participants have valid shares of P_i that they use to compute the polynomials $f_i(x)$, $g_i(x)$ and $h_i(x)$ where $f_i(-1) = x_{1i}$, $f_i(-2) = y_{1i}$, $g_i(-1) = x_{2i}$, $g_i(-2) = y_{2i}$, and $h_i(0) = z_i$.

For this purpose, for a set Q of $(t+1)$ valid shares, a unique value x_{1i} (respectively x_{2i}, y_{1i}, y_{2i}, z_i) is computed with the Lagrange interpolation as $x_{1i} = \sum_{j\in Q} \lambda_{1j} sf_{ij}$, $x_{2i} = \sum_{j\in Q} \lambda_{1j} sg_{ij}$, $y_{1i} = \sum_{j\in Q} \lambda_{2j} sf_{ij}$, $y_{2i} = \sum_{j\in Q} \lambda_{2j} sg_{ij}$, $z_i = \sum_{j\in Q} \lambda_{0j} sh_{ij}$ where $\lambda_{0j} = \prod_{k\in Q, k\neq j} \frac{-k}{j-k}$, $\lambda_{1j} = \prod_{k\in Q, k\neq j} \frac{-1-k}{j-k}$, $\lambda_{2j} = \prod_{k\in Q, k\neq j} \frac{-2-k}{j-k}$ are the coefficient of Lagrange. Thus, from these shares x_1 (respectively x_2, y_1, y_2, z) can be generated as $x_1 = \sum_{i\in QUAL} x_{1i} = \sum_{i\in QUAL} \sum_{j\in Q} \lambda_{1j} sf_{ij} = \sum_{j\in Q} \lambda_{1j} \sum_{i\in QUAL} sf_{ij} = \sum_{j\in Q} \lambda_{1j} sf_j$ (respectively $x_2 = \sum_{j\in Q} \lambda_{1j} sg_j$, $y_1 = \sum_{j\in Q} \lambda_{2j} sf_j$, $y_2 = \sum_{j\in Q} \lambda_{2j} sg_j$, $z = \sum_{j\in Q} \lambda_{0j} sh_j$). For any authorized set Q of participants, the private key $sk = (x_1, x_2, y_1, y_2, z)$ is unique.

Theorem 2. *Our DMKG protocol satisfies the security requirement (C2). Only valid shares of honest participants who check successfully the shares verification are used to reconstruct the private key sk.*

Proof. The validity of the shares (sf_i, sg_i, sh_i) submitted by the participant P_i with $i \in QUAL$ can be checked as follows:

$$\begin{cases} g_1^{sf_i} g_2^{sg_i} = g_1^{\sum_{i\in QUAL} sf_{ij}} g_2^{\sum_{i\in QUAL} sg_{ij}} = \prod_{i\in QUAL} g_1^{sf_{ij}} g_2^{sg_{ij}} = \prod_{i\in QUAL} \prod_{k=0}^{t} (A_{ik})^{j^k} \\ g_1^{sh_i} = g_1^{\sum_{i\in QUAL} sh_{ji}} = \prod_{i\in QUAL} g_1^{sh_{ij}} = \prod_{i\in QUAL} \prod_{k=0}^{t} (A'_{ik})^{j^k} \end{cases}$$

Therefore, only valid shares (sf_i, sg_i, sh_i) submitted by honest participants who passed successfully the check are used.

Theorem 3. *Our DMKG protocol satisfies the security requirement (C3). After the Complaint Management phase, all qualified participants compute the same value of the public key* $pk = (c_1, c_2, c_3) = g_1{}^{x_1} g_2{}^{x_2}, g_1{}^{y_1} g_2{}^{y_2}, g_1{}^{z})$, *where* $sk = (x_1, x_2, y_1, y_2, z)$ *is the unique secret key guaranteed by Theorem 1.*

Proof. Since we did not modify the generation of the public key used in [8], we use the same proof of *Theorem 3* to prove that all qualified participants from the *Complaint Management* phase compute the same value of the public key pk. More details in [8].

Theorem 4. *Our DMKG protocol satisfies the security requirement (C4). The values* x_1, x_2, y_1, y_2 *and* z *of the secret key sk are uniformly distributed in* $\mathbb{Z}_q$.

Proof. At the end of the *Complaint Management* phase, only honest participants are in the set $QUAL$. Each participant P_i, $i \in QUAL$ has already selected a secret value x_{1i} (respectively x_{2i}, y_{1i}, y_{2i}, z_i) randomly in $\mathbb{Z}_q$. As x_1 (respectively x_2, y_1, y_2, z) is defined as the sum of all x_{1i} (respectively x_{2i}, y_{1i}, y_{2i}, z_i) then it can be guaranteed that x_1 (respectively x_2, y_1, y_2, z) is randomly chosen in $\mathbb{Z}_q$. At the end of the *Distribution* phase of our DMKG protocol, these values are already chosen and uniformly distributed via VMSS protocol. Neither the values x_{1i} (respectively x_{2i}, y_{1i}, y_{2i}, z_i) nor the set $QUAL$ change later. Thereby, as $x_1 = \sum_{i \in QUAL} x_{1i}$, then x_1 is uniformly distributed in $\mathbb{Z}_q$. In the same way, x_2, y_1, y_2 and z are also uniformly distributed.

Secrecy Proof. The secrecy of the DMKG is formally expressed by a simulator. Since we have kept the same simulator used in [8], we also keep the same proof of secrecy. We hence refer the reader to the aforementioned publication for further details.

6 Discussion and Comparison

The comparison of our DMKG protocol with BTSOF's DMKG protocol is summarized in Table 1. It takes into consideration (1) the communication model: private or public channels, (2) the computational complexity (3) the security of the DMKG protocol in the sense that the secrecy and correctness requirements are satisfied, (4) the efficiency of the complaint management strategy in the sense that malicious participants are identified and honest participants are not forced to reveal their shares.

The DMKG protocol used in [8] uses private channels to send the shares between participants. Also, it uses a complaint management strategy that is not efficient and does not clearly identify the dishonest participants. The strategy can be briefly described as follow: At the end of the *Distribution and verification* phase and specifically in the *Step 2.h* and *Step 2.i*, if the number of complaints against a participant P_i is superior than $(t-1)$, then P_i is disqualified. Otherwise, for each complaint from P_j against P_i, P_i reveals the shares sent to the participant P_j. In this case, if the revealed values do not check Eq. 2, then the participant P_i is disqualified. This strategy is not effective because in some cases it may not eliminate dishonest participants. Indeed, P_i can send valid shares to P_j but P_j pretends that these shares are invalid to force P_i to reveal his valid shares. In this case, the dishonest participant P_j will not be disqualified. Another case, P_i can cheat and send invalid shares to P_j. Afterwards, during the management of P_j's complaint against P_i, he reveals valid shares. He will not be disqualified. Otherwise, in the *Step 2.e*, all the participants must verify all broadcasted public information $E_{\tau i}$ for $\tau = 1, 2, 3, 4$ of each participant P_i. This verification is costly. Its complexity is $O(kn^2)$ where k is a security parameter and n is the number of participants.

In our DMKG protocol, there is no private channel between participants. It uses only public channels. All the shares are encrypted. Also, the protocol uses

a complaint management strategy that does not depend of the number of complaints reported against a participant to disqualify him. If there is a complaint from P_j against P_i then both participants have to prove their honesty without revealing the values of their shares. Moreover, as previously described, this strategy is more efficient. It clearly identifies the malicious participants. In fact, thanks to the *Complaint Management* phase, the 2 cases described previously are identified and treated. (1) a participant who falsely pretends receiving invalid shares is disqualified and (2) a participant that sends invalid shares is disqualified too. In another hand, in our protocol there is no need to verify all broadcasted information of all participants. The verification is done in the *Complaint management* phase. It is executed only if there is at least one complaint between participants. Thus, the computational complexity is $O(kn)$ and it depends on the number of complaints reported by participants.

Table 1. Comparison of our DMKG protocol with BTSOF's DMKG protocol.

Criteria	BTSOF's DMKG protocol	Our DMKG protocol
Communication Model	Public and private	Public
Computational complexity	$O(kn^2)$	$O(kn)$
Full security	Yes	Yes
Complaint management Efficiency	No	Yes

7 Conclusion

In this paper, we come up with a Distributed Multi-Key Generation (DMKG) protocol based on the DMKG protocol of BTSOF with a new complaint management strategy. The main idea is to avoid honest participants publicly revealing their shares and to efficiently identify dishonest participants. This strategy relies on the *Disputation* strategy proposed in [11] and has been extended to suit the multi secret sharing context. We note that we use only public channels and that all the shares of secrets are encrypted. The proposed DMKG protocol is not limited to Blockchain traceability. It can be used as a building block in several domains such as electronic voting and any identity-based cryptography.

In this work, we focused on the generation of the traceable private key sk. In future works, we intend to use this contribution for the generation of the traceable public key and managing all the complaints between participants. We are working on implementing our protocol in order to validate our contribution with experimental results.

References

1. Biswas, A.K., Dasgupta, M., Ray, S., Khan, M.K.: A probable cheating-free (t, n) threshold secret sharing scheme with enhanced blockchain. Comput. Electr. Eng. **100**, 107925 (2022)
2. Blakley, G.R.: Safeguarding cryptographic keys. In: Managing Requirements Knowledge, International Workshop on, pp. 313–313. IEEE Computer Society (1979)
3. Canetti, R., Gennaro, R., Jarecki, S., Krawczyk, H., Rabin, T.: Adaptive security for threshold cryptosystems. In: Wiener, M. (ed.) CRYPTO 1999. LNCS, vol. 1666, pp. 98–116. Springer, Heidelberg (1999). https://doi.org/10.1007/3-540-48405-1_7
4. Cramer, R., Shoup, V.: A practical public key cryptosystem provably secure against adaptive chosen ciphertext attack. In: Krawczyk, H. (ed.) CRYPTO 1998. LNCS, vol. 1462, pp. 13–25. Springer, Heidelberg (1998). https://doi.org/10.1007/BFb0055717
5. Franklin, M., Yung, M.: Communication complexity of secure computation. In: Proceedings of the 24th Annual ACM Symposium on Theory of Computing, pp. 699–710 (1992)
6. Gennaro, R., Jarecki, S., Krawczyk, H., Rabin, T.: Secure distributed key generation for discrete-log based cryptosystems. In: Stern, J. (ed.) EUROCRYPT 1999. LNCS, vol. 1592, pp. 295–310. Springer, Heidelberg (1999). https://doi.org/10.1007/3-540-48910-X_21
7. Kiamari, N., Hadian, M., Mashhadi, S.: Non-interactive verifiable LWE-based multi secret sharing scheme. Multimedia Tools Appl. pp. 1–13 (2022). https://doi.org/10.1007/s11042-022-13347-4
8. Ma, T., Xu, H., Li, P.: A blockchain traceable scheme with oversight function. In: Meng, W., Gollmann, D., Jensen, C.D., Zhou, J. (eds.) ICICS 2020. LNCS, vol. 12282, pp. 164–182. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-61078-4_10
9. Ma, T., Xu, H., Li, P.: Skyeye: a traceable scheme for blockchain. Cryptology ePrint Archive (2020)
10. Ma, T., Xu, H., Li, P.: A traceable scheme for consortium blockchain. In: 2021 IEEE 9th International Conference on Smart City and Informatization (ISCI), pp. 39–46. IEEE (2021)
11. Neji, W., Blibech, K., Ben Rajeb, N.: Distributed key generation protocol with a new complaint management strategy. Secur. Commun. Netw. **9**(17), 4585–4595 (2016)
12. Pakniat, N., Noroozi, M., Eslami, Z.: Distributed key generation protocol with hierarchical threshold access structure. IET Inf. Secur. **9**(4), 248–255 (2015)
13. Pedersen, T.P.: Non-interactive and information-theoretic secure verifiable secret sharing. In: Feigenbaum, J. (ed.) CRYPTO 1991. LNCS, vol. 576, pp. 129–140. Springer, Heidelberg (1992). https://doi.org/10.1007/3-540-46766-1_9
14. Pedersen, T.P.: A threshold cryptosystem without a trusted party. In: Davies, D.W. (ed.) EUROCRYPT 1991. LNCS, vol. 547, pp. 522–526. Springer, Heidelberg (1991). https://doi.org/10.1007/3-540-46416-6_47
15. Schindler, P., Judmayer, A., Stifter, N., Weippl, E.: Distributed key generation with ethereum smart contracts. In: CIW'19: Cryptocurrency Implementers' Workshop (2019)

16. Shalini, I., Sathyanarayana, S., et al.: A comparative analysis of secret sharing schemes with special reference to e-commerce applications. In: 2015 International Conference on Emerging Research in Electronics, Computer Science and Technology (ICERECT), pp. 17–22. IEEE (2015)
17. Shamir, A.: How to share a secret. Commun. ACM **22**(11), 612–613 (1979)
18. Shil, A.B., Blibech, K., Robbana, R., Neji, W.: A new pvss scheme with a simple encryption function. arXiv preprint arXiv:1307.8209 (2013)
19. Yang, C.C., Chang, T.Y., Hwang, M.S.: A (t, n) multi-secret sharing scheme. Appl. Math. Comput. **151**(2), 483–490 (2004)
20. Zhou, X.: Threshold cryptosystem based fair off-line e-cash. In: 2008 2nd International Symposium on Intelligent Information Technology Application, vol. 3, pp. 692–696. IEEE (2008)